

Sequenzanalyse 2

by Jens Stoye

Summer semester 2026

Lecture notes by Liliana Sanfilippo

April 2026

Contents

1	Sequence Comparison Beyond Edit Distance	5
1.1	The q-gram Distance	5
1.1.1	Practical Computation	6
1.1.2	Choice of q	7
1.1.3	Efficiency	7
1.1.4	Relation to the Edit Distance	7
1.2	The Maximal Matches Distance	7
2	Practical Aspects of de Bruijn Graphs	9
	Backmatter	i
	Definitionen	i
	Figures	iii
	Index	v
	References	v

CHAPTER 1

Sequence Comparison Beyond Edit Distance

The **edit distance** (or **alignment distance**) is one of the most fundamental ways of quantifying the dissimilarity of two sequences x and y over a finite alphabet Σ . It is based on the cost of an alignment $A = (A_1, A_2, \dots, A_n)$, defined as the sum of the costs of A 's columns, i.e.,

$$\text{cost}(A) = \sum_{i=1}^n \text{cost}(A_i),$$

where the cost of alignment column $A_i = \begin{pmatrix} a_i \\ b_i \end{pmatrix}$, $a_i, b_i \in \Sigma$, in the simplest (unit cost) case is 0 for a match column ($a_i = b_i$), and 1 for a mismatch column ($a_i \neq b_i$) or an indel column ($a_i = -$ or $b_i = -$). Then we have:

Definition 1.1 alignment distance

The alignment distance of two sequences $x, y \in \Sigma^*$ is defined as $d(x, y) := \min \{\text{cost}(A) : A \in \mathcal{A}^* \text{ is an alignment of } x \text{ and } y\}$.

The corresponding computational problem is as follows:

Problem 1.1 Alignment Problem

For two given strings $x, y \in \Sigma^*$ and a given cost function, find the alignment distance of x and y and one or all optimal alignment(s).

From the “Sequence Analysis 1” lecture we know that this problem can be solved in $O(mn)$ time using a simple and elegant dynamic programming algorithm, where $m = |x|$ and $n = |y|$ are the two sequence lengths.

However, there are situations in which edit distance and alignments are not the perfect model to compare sequences. In the following two sections we will study two interesting alternatives.

Hier das über die Block moves einfügen.

1.1 | The q-gram Distance

Edit distances emphasize the correct order of the characters in a string. If, however, a large block of a text is moved somewhere else (e.g. two paragraphs of a text are exchanged), the edit distance will be high, although from a certain standpoint, the texts are still quite similar. A more appropriate notion of distance can be defined in several ways. Here we introduce the q-gram distance d_q , which is actually a pseudo-metric: There exist strings $x \neq y$ with $d_q(x, y) = 0$.

Remember that for $q \in \mathbb{N}$, a **q-gram** is a string of length q . Wir reden hier über überlappende q-Gramme!

Definition 1.2 occurrence count

Let $x \in \Sigma^m$ and choose $q \in [1, m]$. The occurrence count of a q-gram $z \in \Sigma^q$ in x is the number

$$\text{Occ}(x, z) := |\{i \in [1, m - q + 1] : x[i \dots i + q - 1] = z\}|$$

Definition 1.3 q-gram profile

Let $\sigma = |\Sigma|$. The q-gram profile of x is a σ^q -element vector $p_q(x) := (p_q(x)_z)_{z \in \Sigma^q}$, indexed by all q-grams, defined by $p_q(x)_z := \text{Occ}(x, z)$

Definition 1.4 q-gram distance

The q-gram distance of $x \in \Sigma^m$ and $y \in \Sigma^n$ is defined for $1 \leq q \leq \min\{m, n\}$ as

$$d_q(x, y) := \sum_{z \in \Sigma^q} |p_q(x)_z - p_q(y)_z|$$

Beispiel 1.1 Consider $\Sigma = A, B$, $x = ABAA$, $y = ABAB$, $z = AABA$, and $q = 2$. Using lexicographic order (AA, AB, BA, BB) among the q-grams, we have $p_2(x) = (1, 1, 1, 0)$, $p_2(y) = (0, 2, 1, 0)$ and $p_2(z) = (1, 1, 1, 0)$. Therefore we obtain $d_2(x, y) = 2$ and $d_2(x, z) = 0$ (although $x \neq z$). Hier bsp. tabelle

Theorem 1.1 The q-gram distance d_q is a pseudo-metric.

Proof. It fulfills nonnegativity, symmetry and the triangle inequality (exercise).

1.1.1 | Practical Computation

If the q-gram profile of a string $x \in \Sigma^m$ is (i.e., if it does not contain mostly zeros), it makes sense to implement it as an array $p[0 \cdots \sigma^q - 1]$, where $p[i]$ counts the number of occurrences of the i -th q-gram in some arbitrary but fixed (e.g. lexicographic) order.

Definition 1.5 ranking function**Definition 1.6 ranking algorithm****Definition 1.7 unranking function****Definition 1.8 unranking algorithm**

Of course, we are interested in an efficient (un-)ranking algorithm for the set of all q-grams over Σ . A classical one is to first define a ranking function r_Σ on the characters (alphabet encoding) and then extend it to a ranking function r on the q-grams by interpreting them as base- σ numbers. There are two possibilities to number the positions of a q-gram: From q to 1 falling or from 1 to q rising, as shown in the Example 1.2.

Beispiel 1.2 Assume the alphabet $\Sigma = A, C, G, T$ and the alphabet encoding $r_\Sigma(A) = 0$, $r_\Sigma(C) = 1$, $r_\Sigma(G) = 2$, and $r_\Sigma(T) = 3$. For $q = 5$, the two different possibilities to rank the q-gram $z = ATACG$ are:

1. Falling ranking

$$r(ATACG) = \dots$$

2. Rising ranking

$$r(ATACG) = \dots$$

The first numbering of positions in (a) corresponds naturally to the ordering of positional number systems; in the decimal number 23, the first 2 has positional value (weight) $10^1 = 10$ whereas the 3 has the lower weight of $10^0 = 1$. For texts, however, the second numbering in (b) is more natural since we expect lower position numbers on the left

side of higher position numbers.

In general, we see that $z \in \Sigma^q$ has rank

$$(a) : r(z) = \sum_{i=1}^q r_{\Sigma}(z[i]) \cdot \sigma^{q-i}, (b) : r(z) = \sum_{i=1}^q r_{\Sigma}(z[i]) \cdot \sum_{j=1}^{i-1}.$$

Whichever numbering system we use, we have to do it consistently.

For each of the two ways to rank a q-gram, there is a corresponding unranking method. Both methods work with integer division and remainder. The one for the falling ranking function uses a dynamic divisor and determines the characters of the q-gram based on the integer results, the other one for the rising ranking function uses a fixed divisor and determines the characters based on the remainder

1.1.2 | Choice of q

1.1.3 | Efficiency

1.1.4 | Relation to the Edit Distance

1.2 | The Maximal Matches Distance

Notiz

Practical Aspects of de Bruijn Graphs

de bruijn subgraph $DBG(s,k) = (V,E)$

V 0 alle k -mere von s

Wie viele k -mere hat eine Sequenz der Länge n sollte man wissen: $n-k+1$

info: für jeden DB Graph gibt es immer einen Eulerpfad

Theorem 2.1 Theorem 2.1 auf deutsch: # worte mit gleichen $(k+1)$ -mer Profil wie s = # Euler-Pfade in $DBG(s,k)$

Kommentar 2.1 man könnte bei bsp 2.1. denken hey bei a kann es nur einen EP geben, weil es kein Kreis ist, und bei 2b mus es dann mehrere geben, eil da eine Schleife drin ist, abre NEIN der Pfad ist die Reihenfolge der Knoten und nicht der Kanten, also gibt es nur einen (visualisieren) Man kann nicht unterscheiden, ob man zuerst die obere oder untere Kante der Schleife zuerst gegangen ist. Gibt also nur eine sequenz mit em $k+1$ meer profil. Denn die Seuenz wird ja aus den Knoten gemacht, nicht den Kanten...sozusagen.

Backmatter

Definitionen

q-gram . 5

alignment distance The alignment distance of two sequences $x, y \in \Sigma^*$ is defined as $d(x, y) := \mincost(A) : A \in \mathcal{A}^*$ is an alignment. 5

edit distance test. 5

occurrence count Let $x \in \Sigma^m$ and choose $q \in [1, m]$. The occurrence count of a q-gram $z \in \Sigma^q$ in x is the number

$$Occ(x, z) := |\{i \in [1, m - q + 1] : x[i \dots i + q - 1] = z\}|$$

. 5

q-gram distance The q-gram distance of $x \in \Sigma^m$ and $y \in \Sigma^n$ is defined for $1 \leq q \leq \min\{m, n\}$ as

$$d_q(x, y) := \sum_{z \in \Sigma^q} |p_q(x)_z - p_q(y)_z|$$

. 6

q-gram profile Let $\sigma = |\Sigma|$. The q-gram profile of x is a σ^q -element vector $p_q(x) := (p_q(x)_z)_{z \in \Sigma^q}$, indexed by all q-grams, defined by $p_q(x)_z := Occ(x, z)$. 6

ranking algorithm . 6

ranking function . 6

unranking algorithm . 6

unranking function . 6

List of Figures